

ARCHITECTURE.md — Barbearia Agendamento Web

1. Visão Geral

Sistema de agendamento online para barbearia, com dois perfis de usuário (cliente e administrador), cálculo dinâmico de slots disponíveis e notificações por e-mail.

Item	Valor
Framework	Django 5.1+
Linguagem	Python 3.x
DB Desenvolvimento	SQLite (db.sqlite3)
DB Produção	PostgreSQL (via dj_database_url)
Servidor WSGI	Gunicorn
Arquivos Estáticos	WhiteNoise
Deploy	Render.com (render.yaml)
Localização	pt-BR / America/Sao_Paulo
Package Manager	pip (requirements/)

2. Estrutura de Diretórios

```
barbearia-agendamento-web/
├── apps/
│   ├── core/           # Autenticação, dashboards, mixins de permissão
│   ├── clientes/       # CRUD de clientes
│   ├── profissionais/  # CRUD de profissionais + grade de horários
│   ├── servicos/       # CRUD de serviços (nome, duração, preço)
│   ├── agendamentos/   # Lógica principal: slots, reservas, cancelamento
│   └── notificacoes/   # Envio de e-mails e log de notificações
├── config/
│   ├── settings/
│   │   ├── base.py     # Configurações compartilhadas
│   │   ├── dev.py      # DEBUG=True, e-mail no console
│   │   └── prod.py     # HTTPS, cookies seguros
│   ├── urls.py         # Roteador raiz
│   └── wsgi.py
├── templates/          # Templates Django por app
├── static/css/         # Estilos globais
├── requirements/
│   ├── base.txt        # Dependências comuns
│   ├── dev.txt
│   └── prod.txt
```

```
| render.yaml          # Serviços web + cron no Render.com
| manage.py
| .env.example         # Variáveis de ambiente documentadas
```

3. Pontos de Entrada & Fluxo

Inicialização

`manage.py` → `config.wsgi:application` (produção via Gunicorn)

Roteamento Raiz (`config/urls.py`)

Prefixo	App
(raiz)	<code>apps.core</code>
<code>clientes/</code>	<code>apps.clientes</code>
<code>profissionais/</code>	<code>apps.profissionais</code>
<code>servicos/</code>	<code>apps.servicos</code>
<code>agendamentos/</code>	<code>apps.agendamentos</code>
<code>admin/</code>	Django Admin

Fluxo Principal de Agendamento

```
Cliente (telefone) → ClienteLoginView → sessão com cliente_id
→ AgendamentoCreateView → availability_api_view (JSON)
→ AvailabilityService.get_slots() → cache (5 min)
→ Agendamento.save() → signal → enviar e-mail de confirmação
```

Fluxo de Lembrete (Cron)

```
Render cron (11:00 UTC diário) → manage.py enviar_lembretes
→ NotificacaoService.enviar_lembrete_dia() → Gmail SMTP → NotificacaoLog
```

4. Padrões de Arquitetura

Estilo: MVT (Model-View-Template) Django padrão, com camada de serviço explícita em `agendamentos/services.py` e `notificacoes/services.py`.

Responsabilidades por Camada

Camada	Localização	Responsabilidade
Model	<code>*/models.py</code>	Entidades, validações, constraints de DB
View	<code>*/views.py</code>	Orquestração de request/response, autenticação
Service	<code>*/services.py</code>	Lógica de negócio isolada (slots, e-mails)
Form	<code>*/forms.py</code>	Validação de entrada do usuário
Signal	<code>agendamentos/signals.py</code>	Efeitos colaterais desacoplados (cache, notificações)
Mixin	<code>core/mixins.py</code>	Controle de acesso reutilizável por CBV

Autenticação Dual

Perfil	Mecanismo	Backend
Cliente	Sessão por telefone (<code>cliente_id</code>)	<code>TelefoneBackend</code> (custom)
Admin	Django auth por e-mail	<code>AdminEmailBackend</code> (custom)

Os dois sistemas são **independentes** — clientes não possuem `User` Django. Acesso protegido pelos mixins `ClienteRequiredMixin` e `AdminRequiredMixin`.

Disponibilidade com Cache

`AvailabilityService` calcula slots livres considerando: horário de trabalho semanal (`ProfessionalSchedule`), intervalo de almoço, duração do serviço e agendamentos existentes. Resultado cacheado por 5 minutos; invalidado via signals em criação/edição/exclusão de agendamentos.

5. Configurações & Infra

Variáveis de Ambiente Essenciais

Variável	Descrição
<code>SECRET_KEY</code>	Chave secreta Django
<code>DEBUG</code>	<code>True</code> (dev) / <code>False</code> (prod)
<code>DATABASE_URL</code>	URL PostgreSQL (prod) ou SQLite (dev)
<code>ALLOWED_HOSTS</code>	Ex: <code>.onrender.com,localhost</code>
<code>TIME_ZONE</code>	Default: <code>America/Sao_Paulo</code>
<code>EMAIL_HOST_USER</code>	Conta Gmail para envio
<code>EMAIL_HOST_PASSWORD</code>	App Password Gmail
<code>DEFAULT_FROM_EMAIL</code>	Endereço remetente
<code>BARBEARIA_NOME</code>	Nome exibido nos e-mails

Variável	Descrição
DJANGO_SUPERUSER_*	Criação automática de superuser via <code>bootstrap</code>

Deploy (Render.com — `render.yaml`)

Serviço	Tipo	Comando
pi-barbearia	Web (free)	<code>gunicorn config.wsgi:application</code>
lembrete-agendamentos	Cron (0 11 * * *)	<code>manage.py enviar_lembretes</code>

- Build: `pip install -r requirements/prod.txt && python manage.py collectstatic --noinput`
- Pre-deploy: `python manage.py migrate`
- Banco: PostgreSQL gerenciado pelo Render, injetado via `DATABASE_URL`

Seleção de Banco por Ambiente

```
if os.getenv("RENDER_SERVICE_TYPE") == "web" and not
os.getenv("RENDER_BUILD_ID"):
    DATABASES = {"default": dj_database_url.config(...)} # PostgreSQL
else:
    DATABASES = {"default": {"ENGINE": "sqlite3", ...}} # SQLite
```

6. Dependências Críticas

Pacote	Função
Django>=5.1	Framework principal
gunicorn	Servidor WSGI de produção
whitenoise	Servir arquivos estáticos sem nginx
dj-database-url	Parse de <code>DATABASE_URL</code>
psycopg[binary]	Driver PostgreSQL
python-decouple	Leitura de variáveis de ambiente / <code>.env</code>

7. Testes & Qualidade

- **Framework:** `unittest` Django (`TestCase`) em `*/tests.py`
- **Apps com testes:** `clientes`, `profissionais`, `servicos`, `agendamentos`, `notificacoes`
- **Lint/Format:** [NÃO DETECTADO] — sem configuração de `flake8`, `black` ou `ruff`
- **Pre-commit hooks:** [NÃO DETECTADO]
- **CI/CD:** [NÃO DETECTADO] — sem `.github/workflows/`

- Execução: `python manage.py test`

8. Limitações & Débitos Técnicos

Outros Débitos

Item	Descrição
Sem CI/CD	Testes não são executados automaticamente no push
Sem lint configurado	Qualidade de código não é verificada por tooling
<code>secret_key.txt</code> no repositório	Arquivo gerado localmente; verificar <code>.gitignore</code>
Cache em memória local	<code>django.core.cache.backends.locmem</code> — não compartilhado entre workers/instâncias
Sem task queue	Notificações síncronas no request cycle; lembretes via cron externo
Clientes sem <code>User</code> Django	Impossibilita uso de funcionalidades nativas (reset de senha, permissions, etc.)
URL duplicada em <code>core/urls.py</code>	<code>meus-agendamentos/</code> mapeada para <code>HomeView</code> em vez de redirecionar corretamente

9. Novas features

- ☐ Incluir botão de concluído na rota `/agendamentos/` (somente para acesso admin)
- Atualmente possui somente opções 'CANCELAR' e 'CONFIRMAR'
- Necessário botão de opção 'CONCLUÍDO', que mudara o status de 'AGENDADO' ou 'CONFIRMADO' para 'CONCLUÍDO', o que significa que o agendamento efetuado foi efetivamente efetuado (cliente atendido).

Como Usar em Futuras Sessões

Antes de responder ou modificar código, leia este arquivo. Ele contém o mapa completo da arquitetura, fluxo e configurações. Não varra o repositório inteiro a menos que seja estritamente necessário para a tarefa. Se precisar de detalhes de um módulo, consulte apenas os caminhos listados na seção "Estrutura de Diretórios" ou "Pontos de Entrada".